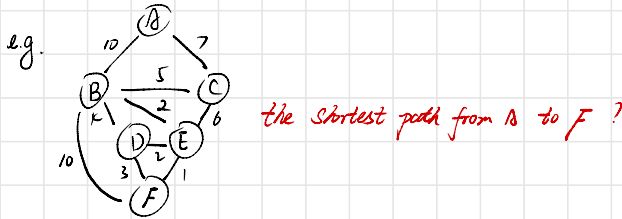


Dijkstra algorithm: See problem 734 Network Delay time in Leetcode as an example.

find minimum path from a starting node to the end node

given a graph.



Code for Dijkstra algorithm:
import heapq → we use heap (min) to maintain a priority queue

```
def Dijkstra (graph, start, end):
```

```
    heap = [ ]
```

```
    heapq.heappush (heap, (0, start))
```

Current path length = 0

```
    path = { }
```

Store the shortest path to each node

```
    while heap:
```

```
        current_node, current_cost = heapq.heappop (heap)
```

```
        if current_node in path:
```

Continue

→ this part is necessary:

```
        path[current_node] = current_cost
```

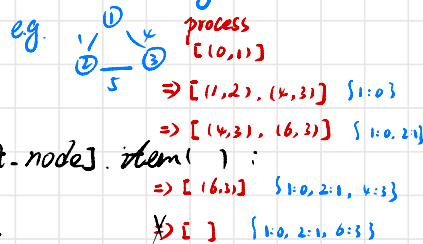
```
        for neighbor, cost in graph[current_node].items():
```

```
            next_cost = current_cost + cost
```

```
            if neighbor not in path:
```

```
                heapq.heappush (heap, next_cost)
```

```
    return path[end]
```



key point:

★ Since we are using priority queue,
the next $\text{Current_Cost} > \text{Current_Cost}$

implication:

1. if current-node has not been heappop before,
i.e. heappop first time, the corresponding Current-Cost
MUST be the minimum time to get to the node.

2. Corresponding, if heappop pop nodes that's already in the
path, the corresponding time would be equal or longer
than the minimum time. Thus, any path afterwards can
be ignored since we already has a better path
with less time.

3. all nodes will be visited

4. time complexity

$$O(E \log V)$$

number of edges
number of vertex

why:

each node will be visited once,
for each visit, we loop through
all neighbors

there could
be multiple
visits, but
except the first
one, we will
continue directly
up to
the number
of neighbors

$$E \quad V \times \underbrace{N}_{\substack{\text{average} \\ \text{number of} \\ \text{neighbors}}} \times \underbrace{\log V}_{\substack{\text{one operation} \\ \text{of heappush}}}$$